



ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR

Membre de
HONORIS UNITED UNIVERSITIES

EMSI Casablanca

Filière Informatique — 4^{ème}IAD

Année universitaire 2025–2026

PROJET DE FIN DE MODULE

Deep Learning

*Conception, implémentation, comparaison
et analyse critique de modèles de Deep Learning
pour données tabulaires, images et séquences*

Auteure : Safaa Hassani

Encadrant : Mr. Batal

Framework : PyTorch 2.10.0+cu128

Environnement : Google Colab — GPU Tesla T4

Résumé

Ce rapport présente les travaux réalisés dans le cadre du projet de fin de module **Deep Learning** à l'**EMSI Casablanca** (filière 4IIR-IAD, 2025–2026).

L'objectif central est d'étudier, d'implémenter et de comparer les trois grandes familles d'architectures neuronales profondes :

- **Partie I** — Perceptron Multicouche (MLP) appliqué à la classification binaire de la qualité des vins rouges (*Wine Quality*, UCI), avec comparaison de méthodes d'initialisation (Gaussienne, Constante, Xavier).
- **Partie II** — Réseau de Neurones Convolutionnel (CNN, architecture LeNet) pour la classification de chiffres manuscrits (MNIST), avec implémentations manuelles des opérations convolutionnelles et comparaison CNN vs MLP.
- **Partie III** — Modèles récurrents (RNN, LSTM, GRU) et système Seq2Seq encodeur-décodeur pour la traduction automatique Anglais→Espagnol, incluant décodage glouton, beam search et évaluation par le score BLEU.

Tous les modèles ont été développés sous **PyTorch** et entraînés sur GPU Tesla T4 via Google Colab.

Mots-clés : Deep Learning, MLP, CNN, LeNet, RNN, LSTM, GRU, Seq2Seq, Teacher Forcing, Beam Search, BLEU, PyTorch, Wine Quality, MNIST.

Table des matières

Résumé	1
Introduction Générale	5
1 Partie I — Perceptron Multicouche (MLP)	6
1.1 Présentation du problème	6
1.2 Dataset : Breast Cancer Wisconsin	6
1.2.1 Variable cible : classification binaire	6
1.3 Analyse Exploratoire des Données	7
1.4 Prétraitement et Normalisation	7
1.4.1 Prévention du <i>data leakage</i>	7
1.4.2 Normalisation Z-score avec StandardScaler	7
1.4.3 Séparation Train / Validation / Test	8
1.4.4 Conversion en tenseurs PyTorch	8
1.5 Architecture du MLP	8
1.5.1 Implémentation avec nn.Sequential	8
1.5.2 Implémentation par classe nn.Module	8
1.5.3 Architecture résumée : $30 \rightarrow 64 \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.3) \rightarrow 2$.	9
1.6 Étude des Paramètres du Réseau	9
1.7 Comparaison des Stratégies d'Initialisation	9
1.8 Régularisation : Dropout et Weight Decay (L2).....	10
1.8.1 Dropout.....	10
1.8.2 Weight Decay (Régularisation L2).....	10
1.9 Early Stopping.....	10
1.10 Entraînement et Exécution sur CPU/GPU	11
1.11 Sauvegarde et Rechargement du Modèle.....	11
1.12 Métriques d'Évaluation et Résultats.....	12
1.12.1 Métriques retenues.....	12
1.12.2 Comparaison avec des modèles classiques de Machine Learning .	12
1.12.3 Analyse critique.....	12
2 Partie II — Réseaux de Neurones Convolutionnels (CNN)	14
2.1 Présentation du problème	14
2.2 Dataset : MNIST	14
2.3 Fondements théoriques des opérations convolutionnelles.....	15
2.3.1 Corrélation croisée 2D	15

2.3.2	Taille de la carte de sortie.....	15
2.3.3	Max Pooling et Average Pooling.....	15
2.3.4	Validation des implémentations manuelles.....	15
2.4	Architecture LeNet.....	16
2.4.1	Propagation des dimensions.....	16
2.4.2	Paramètres du réseau.....	16
2.5	Entraînement.....	17
2.6	Résultats et Analyse.....	17
2.6.1	Courbes d’entraînement.....	17
2.6.2	Comparaison CNN vs MLP sur MNIST.....	17
2.6.3	Visualisation des feature maps.....	17
2.6.4	Analyse critique.....	18
3	Partie III — RNN, LSTM, GRU et Seq2Seq	19
3.1	Présentation du problème.....	19
3.2	Corpus anglais–espagnol.....	19
3.2.1	Génération du corpus.....	19
3.2.2	Statistiques du corpus.....	20
3.2.3	Pipeline de prétraitement.....	20
3.3	Modèle de Langage et Perplexité.....	20
3.3.1	Formalisation.....	20
3.4	Architectures Récurrentes.....	20
3.4.1	RNN Simple.....	20
3.4.2	LSTM (Long Short-Term Memory).....	21
3.4.3	GRU (Gated Recurrent Unit).....	21
3.5	BPTT et Gradient Clipping.....	21
3.6	Comparaison RNN / LSTM / GRU.....	22
3.7	Système Seq2Seq Encodeur–Décodeur.....	22
3.7.1	Architecture.....	22
3.8	Stratégies de Décodage.....	23
3.8.1	Décodage glouton (Greedy Decoding).....	23
3.8.2	Beam Search (k = 3).....	23
3.9	Évaluation : Score BLEU.....	23
3.10	Résultats et Analyse.....	23
3.10.1	Analyse critique.....	23
	Conclusion Générale	25

Liste des tableaux

1	Vue d'ensemble des architectures étudiées	5
1.1	Caractéristiques du dataset Breast Cancer Wisconsin	6
1.2	Distribution des classes — Breast Cancer Wisconsin	7
1.3	Répartition des données — Breast Cancer Wisconsin	8
1.4	Architecture du MLP — Breast Cancer Wisconsin	9
1.5	Comparaison des méthodes d'initialisation	9
1.6	Hyperparamètres d'entraînement — MLP Breast Cancer Wisconsin . .	11
1.7	Métriques de performance — MLP Breast Cancer Wisconsin (test) . .	12
1.8	Comparaison MLP vs modèles classiques — Breast Cancer Wisconsin .	12
2.1	Caractéristiques du dataset MNIST	14
2.2	Propagation des dimensions à travers LeNet (MNIST 28×28).....	16
2.3	Décompte des paramètres de LeNet	16
2.4	Hyperparamètres d'entraînement — LeNet MNIST	17
2.5	Évolution de la perte et de l'accuracy — LeNet sur MNIST	17
2.6	Comparaison MLP vs LeNet (CNN) — MNIST, 10 époques.....	17
3.1	Exemples de paires du corpus EN→ES.....	19
3.2	Statistiques du corpus parallèle EN→ES.....	20
3.3	Hyperparamètres du modèle de langage récurrent.....	22
3.4	Comparaison RNN / LSTM / GRU — Modèle de langage espagnol.....	22
3.5	Hyperparamètres du système Seq2Seq.....	22
3.6	Résultats du système Seq2Seq — corpus EN→ES.....	23
3.7	Bilan des architectures étudiées	25
3.8	Récapitulatif des performances obtenues	25

Introduction Générale

Le **Deep Learning** est aujourd’hui au cœur des avancées majeures en intelligence artificielle : reconnaissance d’images, traitement du langage naturel, traduction automatique, génération de contenu. Ces progrès reposent sur la capacité des réseaux de neurones profonds à apprendre automatiquement des représentations hiérarchiques à partir de données brutes.

Ce projet de fin de module a pour objectif de mettre en pratique les architectures fondamentales étudiées en cours, en les appliquant à des problèmes réels représentatifs de chaque domaine. L’implémentation repose entièrement sur le framework **PyTorch**, qui offre un paradigme de définition dynamique des graphes de calcul (*eager execution*), facilitant le débogage et la flexibilité architecturale.

Les trois familles d’architectures étudiées sont présentées dans le tableau 1, chacune appliquée à un type de données et une tâche distincts.

TABLE 1 – Vue d’ensemble des architectures étudiées

Partie	Architecture	Dataset	Tâche
I	MLP (Perceptron Multicouche)	Breast Cancer Wisconsin	Classification binaire
II	CNN — LeNet	MNIST	Reconnaissance de chiffres
III	RNN / LSTM / GRU / Seq2Seq	Corpus EN→ES	Traduction automatique

L’environnement d’exécution est **Google Colab** avec un GPU **Tesla T4**, PyTorch 2.10.0+cu128, Python 3.10+. Le rapport suit l’organisation des trois notebooks Jupyter produits lors du projet.

1. Partie I – Perceptron Multicouche (MLP)

1.1 Présentation du problème

La **détection du cancer du sein** est un problème de classification binaire supervisée sur des données médicales tabulaires. L'objectif est de prédire, à partir de mesures caractérisant les cellules tumorales, si une tumeur est *bénigne* (Benign) ou *maligne* (Malignant), afin d'assister le diagnostic clinique.

1.2 Dataset : Breast Cancer Wisconsin

Le jeu de données **Breast Cancer Wisconsin** est un jeu de données médical largement utilisé en apprentissage automatique pour l'évaluation des algorithmes de détection du cancer du sein.

TABLE 1.1 – Caractéristiques du dataset Breast Cancer Wisconsin

Propriété	Valeur
Nombre d'observations	569
Nombre de variables	30 caractéristiques numériques
Type des variables	Float64 (toutes continues)
Valeurs manquantes	Aucune
Tâche	Classification binaire

Les 30 caractéristiques décrivent des propriétés géométriques et texturales des noyaux cellulaires (rayon, texture, périmètre, aire, lissé, compacité, concavité, symétrie, dimension fractale) calculées selon trois statistiques : *mean*, *standard error* et *worst* (valeur maximale).

1.2.1 Variable cible : classification binaire

La variable cible est directement binaire :

TABLE 1.2 – Distribution des classes — Breast Cancer Wisconsin

Classe	Étiquette	Effectif	
0	Malin (Malignant)	212	(37.3%)
1	Bénin (Benign)	357	(62.7%)
Total		569	

1.3 Analyse Exploratoire des Données

L'analyse exploratoire a porté sur :

- **Statistiques descriptives** : distribution des 30 variables, détection d'outliers, plages de valeurs très disparates (nécessitant une normalisation obligatoire).
- **Distribution de la cible** : déséquilibre modéré (37 %/63 %) ; le jeu de données reste exploitable sans rééchantillonnage.
- **Matrice de corrélation** : forte corrélation entre les features *mean* et *worst* pour une même propriété géométrique, confirmant la redondance partielle des 30 variables.
- **Visualisation par classe** : les variables de rayon, périmètre et aire montrent les meilleures séparations visuelles entre classes Bénin et Malin.

1.4 Prétraitement et Normalisation

1.4.1 Prévention du *data leakage*

La normalisation est calculée **uniquement** sur l'ensemble d'entraînement et appliquée ensuite aux ensembles de validation et de test. Cette règle stricte (*fit on train, transform all*) évite tout biais d'information du futur vers le passé.

1.4.2 Normalisation Z-score avec StandardScaler

$$x_{\text{norm}} = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}} \quad (1.1)$$

Après normalisation : $\mu_{\text{train}} \approx 0$, $\sigma_{\text{train}} \approx 1$ pour chaque variable.

```

1 scaler = StandardScaler ()
2 X_train = scaler.fit_transform(X_train)    # fit uniquement sur
   train
3 X_val   = scaler.transform(X_val)          # memes parametres
4 X_test  = scaler.transform(X_test)
```

Listing 1.1 – Normalisation sans data leakage

1.4.3 Séparation Train / Validation / Test

TABLE 1.3 – Répartition des données — Breast Cancer Wisconsin

Ensemble	Exemples	Proportion
Entraînement	398	70%
Validation	85	15%
Test	86	15%
Total	569	100%

1.4.4 Conversion en tenseurs PyTorch

```

1 X_train_t = torch.FloatTensor(X_train) # dtype: float32
2 y_train_t = torch.LongTensor(y_train) # dtype: int64

```

Listing 1.2 – Conversion en tenseurs PyTorch

1.5 Architecture du MLP

1.5.1 Implémentation avec nn.Sequential

```

1 model = nn.Sequential(
2     nn.Linear(30, 64),
3     nn.ReLU(),
4     nn.Dropout(0.3),
5     nn.Linear(64, 2)
6 )

```

Listing 1.3 – MLP via nn.Sequential

1.5.2 Implémentation par classe nn.Module

```

1 class MLP(nn.Module):
2     def __init__(self, input_size=30, hidden=64, output_size=2):
3         super(MLP, self).__init__()
4         self.fc1 = nn.Linear(input_size, hidden)
5         self.fc2 = nn.Linear(hidden, output_size)
6         self.relu = nn.ReLU()
7         self.dropout = nn.Dropout(0.3)
8
9         def forward(self, x):
10             x = self.relu(self.fc1(x))
11             x = self.dropout(x)
12             x = self.fc2(x)
13             return x
14
15 model = MLP(input_size=30, hidden=64, output_size=2)

```

Listing 1.4 – MLP personnalisé via nn.Module

1.5.3 Architecture résumée : 30 → 64 → ReLU → Dropout(0.3) → 2

TABLE 1.4 – Architecture du MLP — Breast Cancer Wisconsin

#	Couche	Entrée	Sortie	Paramètres	Activation
1	Linear	30	64	$30 \times 64 + 64 = 1\,984$	ReLU
—	Dropout	64	64	0	—
2	Linear	64	2	$64 \times 2 + 2 = 130$	Softmax
Total				2 114	—

1.6 Étude des Paramètres du Réseau

L'inspection du réseau via `named_parameters()` et `state_dict()` permet de visualiser la forme et le nombre de paramètres de chaque couche :

```

1 for name, param in model.named_parameters():
2     print(f'{name} | forme : {param.shape} | nb : {param.numel}
3         O}')
4 # fc1 . weight | forme : torch.Size([64 , 30]) | nb : 1920
5 # fc1 . bias   | forme : torch.Size([64])       | nb : 64
6 # fc2 . weight | forme : torch.Size([2 , 64])    | nb : 128
7 # fc2 . bias   | forme : torch.Size([2])         | nb : 2

```

Listing 1.5 – Inspection des paramètres via `named_parameters()`

Le `state_dict()` est utilisé pour la sauvegarde et le rechargement du modèle (voir section 1.11).

1.7 Comparaison des Stratégies d'Initialisation

Trois stratégies d'initialisation des poids ont été comparées :

TABLE 1.5 – Comparaison des méthodes d'initialisation

Méthode	Distribution	Commentaire
Gaussienne	$W \sim N(0, 0.01^2)$	Simple ; peut ralentir la convergence si σ mal calibré
Constante	$W = 0.1$	À éviter : tous les neurones apprennent la même chose (symétrie non brisée)
Xavier uniform	$W \sim \frac{6}{S - \tilde{n}} + \frac{6}{\tilde{n} - 1}$	Retenue : maintient la variance stable entre couches

L'initialisation **Xavier uniforme** a été retenue car elle préserve approximativement la variance des activations à travers les couches, évitant la saturation et l'évanouissement des gradients lors des premières époques.

1.8 Régularisation : Dropout et Weight Decay (L2)

1.8.1 Dropout

Le Dropout désactive aléatoirement une fraction $p = 0.3$ des neurones à chaque forward pass durant l'entraînement, forçant le réseau à apprendre des représentations redondantes et ainsi réduire le surapprentissage :

$$\tilde{h}_i = \begin{cases} 0 & \text{avec probabilité } p \\ h_i/(1 - p) & \text{sinon} \end{cases} \quad (1.2)$$

1.8.2 Weight Decay (Régularisation L2)

La régularisation L2 (Weight Decay) pénalise les poids de grande amplitude en ajoutant un terme quadratique à la fonction de perte :

$$L_{\text{total}} = L_{\text{CE}} + \lambda \sum_i \|W_i\|_F^2 \quad (1.3)$$

Dans PyTorch, ce terme est intégré directement dans l'optimiseur Adam via le paramètre `weight_decay` :

```
1 optimizer = torch.optim.Adam(model.parameters(),
2                               lr=0.001,
3                               weight_decay=1e-4)
```

Listing 1.6 – Optimiseur Adam avec Weight Decay

1.9 Early Stopping

L'**Early Stopping** interrompt l'entraînement lorsque la perte de validation ne s'améliore plus pendant un certain nombre d'époques consécutives (*patience*), évitant ainsi le surapprentissage sans nécessiter de fixer le nombre d'époques a priori :

```
1 best_val_loss = float('inf')
2 patience, counter = 10, 0
3
4 for epoch in range(num_epochs):
5     # ... entraînement ...
6     if val_loss < best_val_loss:
7         best_val_loss = val_loss
8         counter = 0
9         torch.save(model.state_dict(), "best_model.pth")
10    else:
11        counter += 1
```

```

12         if counter >= patience :
13             print (f" Early stopping a l'epoque {epoch +1}")
14             break

```

Listing 1.7 – Logique d'Early Stopping

1.10 Entraînement et Exécution sur CPU/GPU

TABLE 1.6 – Hyperparamètres d'entraînement — MLP Breast Cancer Wisconsin

Hyperparamètre	Valeur
Optimiseur	Adam
Taux d'apprentissage	0.001
Weight Decay (λ)	10^{-4}
Dropout	0.3
Fonction de perte	CrossEntropyLoss
Initialisation	Xavier uniforme
Early Stopping patience	10 époques
Device	GPU Tesla T4 (CUDA)

Le modèle et les tenseurs sont déplacés sur le GPU via `.to(device)`, permettant d'accélérer significativement les calculs matriciels :

```

1 device = torch.device("cuda" if torch.cuda.is_available() else
2     "cpu")
3 model = model.to(device)
4 X_train_t = X_train_t.to(device)
5 y_train_t = y_train_t.to(device)
6 # Device utilise : cuda (Tesla T4)

```

Listing 1.8 – Exécution sur CPU/GPU

1.11 Sauvegarde et Rechargement du Modèle

Le meilleur modèle (celui minimisant la perte de validation) est sauvegardé via `state_dict()` et rechargé pour l'évaluation finale :

```

1 # Sauvegarde
2 torch.save(model.state_dict(), "best_model.pth")
3
4 # Rechargement
5 model.load_state_dict(torch.load("best_model.pth"))
6 model.eval()

```

Listing 1.9 – Sauvegarde et rechargement du meilleur modèle

1.12 Métriques d'Évaluation et Résultats

1.12.1 Métriques retenues

Cinq métriques complémentaires ont été calculées sur le jeu de test :

- **Accuracy** : proportion de prédictions correctes (toutes classes)
- **Precision** : parmi les cas prédits Malins, proportion réellement malins
- **Recall** : parmi les cas réellement Malins, proportion correctement détectés
- **F1-Score** : moyenne harmonique de la Precision et du Recall
- **Matrice de confusion** : répartition des vrais/faux positifs et négatifs

TABLE 1.7 – Métriques de performance — MLP Breast Cancer Wisconsin (test)

Métrique	Valeur
Accuracy	> 95%
Precision	> 0.94
Recall	> 0.94
F1-Score	> 0.94

1.12.2 Comparaison avec des modèles classiques de Machine Learning

Afin d'évaluer la valeur ajoutée du MLP par rapport aux approches classiques, deux modèles de référence ont été entraînés sur les mêmes données :

TABLE 1.8 – Comparaison MLP vs modèles classiques — Breast Cancer Wisconsin

Modèle	Paramètres/Complexité	Accuracy	F1-Score
Régression Logistique	Linéaire, simple	≈ 95%	≈ 0.94
Forêt Aléatoire	Ensemble, robuste	≈ 96%	≈ 0.95
MLP (notre)	2 114 paramètres	> 95%	> 0.94

1.12.3 Analyse critique

- L'accuracy obtenue par le MLP est **compétitive** avec les modèles classiques sur ce dataset médical, confirmant l'efficacité des perceptrons multicouches sur des données tabulaires bien prétraitées.
- La **Régression Logistique** est le modèle de référence naturel pour ce type de tâche : linéaire, interprétable, peu coûteux. Ses performances proches de celles du MLP suggèrent que la frontière de décision est quasi-linéaire dans cet espace de features normalisées.
- La **Forêt Aléatoire** surpasse légèrement le MLP sur ce petit dataset (569 exemples), ce qui est attendu : les méthodes ensemblistes sont particulièrement robustes en faible volume de données.

-
- La combinaison **Dropout + Weight Decay** réduit efficacement le surapprentissage, visible par la faible divergence entre les courbes train/val.
 - L'**Early Stopping** s'est avéré essentiel sur ce petit dataset : sans lui, le modèle surapprendrait rapidement les 398 exemples d'entraînement.
 - En contexte médical, le **Recall** (sensibilité) est la métrique prioritaire : une fausse négative (Malin classé Bénin) a des conséquences plus graves qu'une fausse positive.

2. Partie II — Réseaux de Neurones Convolutionnels (CNN)

2.1 Présentation du problème

La **reconnaissance de chiffres manuscrits** est une tâche de classification multi-classes sur des données d'image en niveaux de gris. Elle constitue un problème de référence pour évaluer les architectures convolutionnelles.

2.2 Dataset : MNIST

Le jeu de données **MNIST** (*Modified National Institute of Standards and Technology*) est la référence historique de l'évaluation des algorithmes de reconnaissance d'images.

TABLE 2.1 – Caractéristiques du dataset MNIST

Propriété	Valeur
Images d'entraînement	60 000
Images de test	10 000
Résolution	28 × 28 pixels
Canaux	1 (niveaux de gris)
Nombre de classes	10 (chiffres 0 – 9)

Prétraitement appliqué :

- Conversion en tenseur : `transforms.ToTensor()` $\Rightarrow$ valeurs $\in [0, 1]$
- Normalisation : `transforms.Normalize((0.5,), (0.5,))` $\Rightarrow$ valeurs $\in [-1, 1]$
- Batches de taille 64, shuffle activé sur l'entraînement

2.3 Fondements théoriques des opérations convolutionnelles

2.3.1 Corrélation croisée 2D

L'opération centrale d'un CNN est la **corrélation croisée 2D** (souvent appelée convolution par abus de langage) :

$$(X \star K)_{i,j} = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} X_{i+m, j+n} \cdot K_{m, n} \quad (2.1)$$

où $X \in \mathbb{R}^{H \times W}$ est la carte d'entrée et $K \in \mathbb{R}^{k_h \times k_w}$ est le noyau.

2.3.2 Taille de la carte de sortie

$$H_{out} = \left\lceil \frac{H_{in} + 2p - k}{s} \right\rceil + 1 \quad (2.2)$$

avec p le *padding*, k la taille du noyau et s le *stride*.

2.3.3 Max Pooling et Average Pooling

$$\text{MaxPool}_{i,j} = \max_{(m,n) \in \mathcal{X}_{i,j}} X_{m,n} \quad (2.3)$$

$$\text{AvgPool}_{i,j} = \frac{1}{k^2} \sum_{(m,n) \in \mathcal{X}_{i,j}} X_{m,n} \quad (2.4)$$

où $\mathcal{R}_{i,j}$ est la région de pooling centrée en (i,j) .

2.3.4 Validation des implémentations manuelles

Les trois opérations (corrélation croisée, max pooling, average pooling) ont été implémentées manuellement en NumPy puis validées par comparaison avec les couches natives PyTorch (`F.conv2d`, `F.max_pool2d`) :

```

1 result_manual = correlation_croisee_2d(input_test, kernel_test
2 )
3 result_torch = F.conv2d(input_torch, kernel_torch).squeeze().
4 numpy ()
5
6 print("Identiques :", np.allclose(result_manual, result_torch))
7
8 # --> Identiques : True

```

Listing 2.1 – Validation corrélation croisée manuelle vs PyTorch

2.4 Architecture LeNet

L'architecture **LeNet-5** (LeCun *et al.*, 1998) a été adaptée pour MNIST.

```
[caption=Implémentation de LeNet pour MNIST] class LeNet(nn.Module):
    def __init__(self):
        super(LeNet, self).__init__()
        self.conv1 = nn.Conv2d(1, 16, 5, 1)
        self.pool1 = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(16, 16, 5, 1)
        self.pool2 = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = torch.sigmoid(self.conv1(x))
        x = self.pool1(x)
        x = torch.sigmoid(self.conv2(x))
        x = self.pool2(x)
        x = x.view(-1, 16 * 5 * 5)
        x = torch.sigmoid(self.fc1(x))
        x = torch.sigmoid(self.fc2(x))
        return self.fc3(x)
```

2.4.1 Propagation des dimensions

En appliquant l'équation (2.2) et la formule du pooling à une image 28×28 :

TABLE 2.2 – Propagation des dimensions à travers LeNet (MNIST 28×28)

Couche	Opération	k	p	Sortie
Entrée	—	—	—	$1 \times 28 \times 28$
Conv1	Corrélation croisée	5	2	$6 \times 28 \times 28$
AvgPool1	Average Pooling	2	0	$6 \times 14 \times 14$
Conv2	Corrélation croisée	5	0	$16 \times 10 \times 10$
AvgPool2	Average Pooling	2	0	$16 \times 5 \times 5$
Flatten	—	—	—	400
FC1	Linear	—	—	120
FC2	Linear	—	—	84
FC3	Linear (sortie)	—	—	10

2.4.2 Paramètres du réseau

TABLE 2.3 – Décompte des paramètres de LeNet

Couche	Calcul	Paramètres
Conv1	$6 \times (1 \times 5 \times 5 + 1)$	156
Conv2	$16 \times (6 \times 5 \times 5 + 1)$	2 416
FC1	$400 \times 120 + 120$	48 120
FC2	$120 \times 84 + 84$	10 164
FC3	$84 \times 10 + 10$	850
Total		61 706

2.5 Entraînement

TABLE 2.4 – Hyperparamètres d’entraînement — LeNet MNIST

Hyperparamètre	Valeur
Nombre d’époques	10
Taille de batch	64
Optimiseur	Adam
Taux d’apprentissage	0.001
Fonction de perte	CrossEntropyLoss
Activation (conv)	Sigmoid

2.6 Résultats et Analyse

2.6.1 Courbes d’entraînement

TABLE 2.5 – Évolution de la perte et de l’accuracy — LeNet sur MNIST

Époque	Train Loss	Test Accuracy
1	1.0198	92.16%
2	0.2190	95.51%
3	0.1390	97.07%
4	0.1030	97.67%
5	0.0842	97.63%
6	0.0719	97.73%
7	0.0628	97.98%
8	0.0553	98.35%
9	0.0507	98.31%
10	0.0464	98.41%

2.6.2 Comparaison CNN vs MLP sur MNIST

Pour quantifier l’apport des couches convolutionnelles, un MLP simple (architecture $784 \rightarrow 256 \rightarrow 128 \rightarrow 10$) a été entraîné sur les mêmes images :

TABLE 2.6 – Comparaison MLP vs LeNet (CNN) — MNIST, 10 époques

Modèle	Paramètres	Meilleure Accuracy	Avantage clé
MLP simple	$\approx 235\,000$	$\approx 97\%$	Simplicité
LeNet (CNN)	61 706	98.41%	Partage de poids, invariance

2.6.3 Visualisation des feature maps

L’inspection des 6 cartes de caractéristiques après la première couche convolutionnelle révèle que chaque filtre détecte des motifs visuels distincts (contours, textures

locales), illustrant la capacité du CNN à extraire automatiquement des **représentations hiérarchiques** sans ingénierie manuelle de features.

2.6.4 Analyse critique

- L'**accuracy de 98.41%** en seulement 10 époques démontre l'efficacité de l'architecture LeNet sur MNIST.
- Le **partage de poids** dans les couches convolutionnelles (61 706 paramètres) est plus économique que le MLP ($\approx 235\,000$) tout en offrant de meilleures performances.
- La convergence rapide dès la 1^{ère} époque (92.16%) montre l'efficacité de la représentation spatiale apprise par les filtres.
- L'**invariance par translation** induite par le pooling est un avantage structural majeur des CNN sur les données visuelles.
- L'utilisation de l'activation **Sigmoid** dans les couches convolutionnelles est une limitation par rapport à ReLU (saturation des gradients) ; l'emploi de ReLU améliorerait probablement la convergence.

3. Partie III — RNN, LSTM, GRU et Seq2Seq

3.1 Présentation du problème

Cette partie porte sur la **modélisation de séquences** et la **traduction automatique**. La tâche est la traduction Anglais → Espagnol, un problème non trivial car il implique :

1. L'apprentissage du **lexique** bilingue
2. Le **réordonnement** syntaxique : en anglais l'adjectif précède le nom (« the *red* car »), en espagnol il le suit (« el coche *rojo* »)
3. L'**accord en genre** grammatical (masculin / féminin)

3.2 Corpus anglais-espagnol

3.2.1 Génération du corpus

Un corpus parallèle contrôlé a été généré à partir d'un lexique bilingue de 12 noms (avec genre grammatical) et 10 adjectifs, combinés selon 4 patrons de phrases. Le dédoublonnage produit $\approx 6\,000$ paires uniques.

TABLE 3.1 – Exemples de paires du corpus EN→ES

Anglais	Espagnol
the red car	el coche rojo
a small house	una casa pequeña
i see the black bird	veo el pajarito negro
this is a new book	esto es un libro nuevo
the yellow flower	la flor amarilla

3.2.2 Statistiques du corpus

TABLE 3.2 – Statistiques du corpus parallèle EN→ES

Propriété	Valeur
Paires uniques totales	≈ 6 000
Ensemble d'entraînement	85% (≈ 5 100 paires)
Ensemble de test	15% (≈ 900 paires)
Taille du vocabulaire anglais	35 mots (+ SOS, EOS, PAD)
Taille du vocabulaire espagnol	43 mots (+ SOS, EOS, PAD)
Longueur maximale des séquences	10 tokens

3.2.3 Pipeline de prétraitement

1. **Tokenisation** par séparation sur les espaces
2. **Construction du vocabulaire** avec tokens spéciaux : SOS (0), EOS (1), PAD (2)
3. **Encodage** des phrases en séquences d'indices
4. **Padding** à la longueur maximale (10 tokens)
5. **DataLoader** avec batches de 64, shuffle sur l'entraînement

3.3 Modèle de Langage et Perplexité

3.3.1 Formalisation

Un modèle de langage factorise la probabilité d'une séquence par la règle de chaîne :

$$P(w_1, w_2, \dots, w_T) = \prod_{t=1}^T P(w_t \mid w_1, \dots, w_{t-1}) \quad (3.1)$$

La **perplexité** est la métrique standard des modèles de langage :

$$\text{PPL} = \exp \left(-\frac{1}{T} \sum_{t=1}^T \log P(w_t \mid w_1, \dots, w_{t-1}) \right) \quad (3.2)$$

Une perplexité faible signifie que le modèle prédit le mot suivant avec plus de certitude. Un modèle aléatoire sur un vocabulaire de $|V|$ mots a une perplexité de $|V|$.

3.4 Architectures Récurrentes

3.4.1 RNN Simple

$$h_t = \tanh \left(W_{xh} x_t + W_{hh} h_{t-1} + b_h \right) \quad (3.3)$$

Le RNN simple souffre du problème des **gradients qui s'évanouissent** (ou explosent) lors de la BPTT sur des séquences longues, car le gradient se propage comme un produit de matrices jacobienne dont la norme croît ou décroît exponentiellement avec le temps.

3.4.2 LSTM (Long Short-Term Memory)

Le LSTM (Hochreiter & Schmidhuber, 1997) introduit un vecteur d'état de cellule c_t et trois portes pour contrôler le flux d'information :

$$f_t = \sigma(W_f [h_{t-1}, x_t] + b_f) \quad (\text{porte d'oubli}) \quad (3.4)$$

$$i_t = \sigma(W_i [h_{t-1}, x_t] + b_i) \quad (\text{porte d'entrée}) \quad (3.5)$$

$$\tilde{c}_t = \tanh(W_c [h_{t-1}, x_t] + b_c) \quad (\text{candidat}) \quad (3.6)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (\text{état de cellule}) \quad (3.7)$$

$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o) \quad (\text{porte de sortie}) \quad (3.8)$$

$$h_t = o_t \odot \tanh(c_t) \quad (\text{état caché}) \quad (3.9)$$

3.4.3 GRU (Gated Recurrent Unit)

Le GRU (Cho *et al.*, 2014) simplifie le LSTM avec deux portes :

$$z_t = \sigma(W_z [h_{t-1}, x_t]) \quad (\text{porte de mise à jour}) \quad (3.10)$$

$$r_t = \sigma(W_r [h_{t-1}, x_t]) \quad (\text{porte de réinitialisation}) \quad (3.11)$$

$$\tilde{h}_t = \tanh(W [r_t \odot h_{t-1}, x_t]) \quad (3.12)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (3.13)$$

3.5 BPTT et Gradient Clipping

La **rétropropagation à travers le temps** (BPTT) propage le gradient en arrière à travers chaque pas de temps :

$$\frac{\partial L}{\partial W} = \sum_{t=1} \frac{\partial L_t}{\partial W} = \sum_{t=1} \frac{\partial L_t}{\partial h_t} \prod_{k=t}^{T-1} \frac{\partial h_{k+1}}{\partial h_k} \cdot \frac{\partial h_t}{\partial W} \quad (3.14)$$

La norme du produit de jacobiens se comporte en $(\rho_{\text{spec}})^T$. Le **gradient clipping** borne la norme pour éviter l'explosion :

$$g \leftarrow \frac{c}{\|g\|} \cdot g \quad \text{si } \|g\| > c \quad (3.15)$$

L'expérience montre que l'entraînement du RNN simple **sans** clipping diverge ou oscille, tandis qu'avec clipping ($c = 1.0$) la convergence est stable.

3.6 Comparaison RNN / LSTM / GRU

TABLE 3.3 – Hyperparamètres du modèle de langage récurrent

Hyperparamètre	Valeur
Taille d’embedding	128
Taille état caché	256
Nombre d’époques	6
Optimiseur	Adam (lr = 0.001)
Gradient clipping	1.0

TABLE 3.4 – Comparaison RNN / LSTM / GRU — Modèle de langage espagnol

Architecture	Perplexité finale	Stabilité	Complexité	Nb. paramètres
RNN simple	Élevée	Instable (sans clipping)	Faible	Faible
LSTM	Basse	Bonne	Haute	Élevé
GRU	Basse	Bonne	Moyenne	Moyen

3.7 Système Seq2Seq Encodeur–Décodeur

3.7.1 Architecture

[caption=Encodeur et Décodeur GRU] class Encoder(nn.Module) : def $init_{(self, vocab_size, embed_size=128, hidden_size=256)}$

class Decoder(nn.Module) : def $init_{(self, vocab_size, embed_size=128, hidden_size=256)}$: sup,

$$\text{input}_t = \begin{cases} y_{t-1} & \text{avec probabilité } p = 0.5 \quad (\text{supervision directe}) \\ \hat{y}_{t-1} & \text{avec probabilité } 1 - p \quad (\text{prédiction propre}) \end{cases} \quad (3.16)$$

Le teacher forcing accélère la convergence mais peut créer un décalage entre l’entraînement et l’inférence (*exposure bias*).

TABLE 3.5 – Hyperparamètres du système Seq2Seq

Hyperparamètre	Valeur
Nombre d’époques	40
Taille de batch	64
Taille d’embedding	128
Taille état caché	256
Optimiseur	Adam (lr = 0.001)
Teacher forcing ratio	0.5
Gradient clipping	1.0
Fonction de perte	CrossEntropyLoss (masque PAD)

3.8 Stratégies de Décodage

3.8.1 Décodage glouton (Greedy Decoding)

À chaque pas de temps, le token de plus haute probabilité est sélectionné :

$$\hat{y}_t = \arg \max_{w \in V} P(w \mid \hat{y}_1, \dots, \hat{y}_{t-1}, c) \quad (3.17)$$

Simple et rapide, mais sous-optimal : il peut manquer des séquences globalement meilleures.

3.8.2 Beam Search (k = 3)

Le beam search maintient les k meilleures hypothèses partielles à chaque pas :

$$\text{score}(\mathbf{y}) = \frac{1}{|\mathbf{y}|} \sum_{t=1}^T \log P(y_t \mid y_1, \dots, y_{t-1}, c) \quad (3.18)$$

La normalisation par la longueur évite le biais en faveur des séquences courtes.

3.9 Évaluation : Score BLEU

Le score **BLEU** (*Bilingual Evaluation Understudy*, Papineni *et al.*, 2002) mesure le recouvrement des n-grammes entre la traduction candidate et la référence :

$$\text{BLEU} = \underbrace{\exp \min_{0, 1} \left(1 - \frac{|\text{ref}|}{|\text{cand}|} \right)}_{\text{pénalité de brevité}} \cdot \underbrace{\exp \frac{1}{N} \sum_{n=1}^N \log p_n}_{\text{précision BP}} \quad (3.19)$$

où p_n est la précision des n-grammes d'ordre n (avec *clip* pour éviter le gonflement artificiel).

3.10 Résultats et Analyse

TABLE 3.6 – Résultats du système Seq2Seq — corpus EN→ES

Stratégie de décodage	BLEU moyen	Traductions exactes
Greedy (glouton)	≈ 1.0	≈ 100%
Beam Search (k = 3)	≈ 1.0	≈ 100%

3.10.1 Analyse critique

- Le système Seq2Seq atteint une **correspondance exacte de** ≈ 100% sur ce corpus contrôlé, y compris sur des combinaisons nom-adjectif jamais vues en

- entraînement. Cela prouve que le modèle a appris la **règle compositionnelle** (lexique + réordonnancement + accord en genre) et non une simple mémorisation.
- Ce score élevé est spécifique à la **régularité du corpus** ; sur un corpus réel (vocabulaire vaste, ambiguïté, longues phrases), les performances chuteraient et la limite du **goulot d'étranglement** (un seul vecteur de contexte de taille fixe) deviendrait critique.
 - Sur ce corpus très régulier et ces séquences courtes, **beam search** et décodage glouton coïncident car le modèle est très confiant à chaque pas. Le beam search n'apporte un gain significatif qu'en présence d'incertitude.
 - LSTM et GRU obtiennent des perplexités proches sur ces séquences courtes ; l'écart se creuserait sur de longues dépendances où les portes des LSTM/GRU stabilisent l'apprentissage.
 - La solution naturelle au goulot d'étranglement est le mécanisme d'**attention** (Bahdanau *et al.*, 2014), qui permet au décodeur d'accéder directement à l'ensemble des états de l'encodeur, ouvrant la voie à l'architecture **Transformer**.

Conclusion Générale

Ce projet a permis de mettre en œuvre et d’analyser en profondeur les trois grandes familles d’architectures du Deep Learning, en les appliquant à des jeux de données réels représentatifs de leur domaine naturel.

Bilan comparatif des architectures

TABLE 3.7 – Bilan des architectures étudiées

Architecture	Données adaptées	Points forts	Limites
MLP	Tabulaires structurées	Simple, efficace sur features indépendantes	Ignore structures spatiales/temporelles
CNN (LeNet)	Images, données 2D	Invariance par translation, partage de poids, hiérarchie de features	Moins adapté aux séquences ; convolutions classiques fixent la topologie
RNN LSTM GRU Seq2Seq	/ Séquences (texte, audio, séries temporelles)	Modélise les dépendances temporelles et le contexte	Goulot d’étranglement sur longues séquences ; résolu par l’attention

Récapitulatif des performances

TABLE 3.8 – Récapitulatif des performances obtenues

Partie	Modèle	Dataset	Performance
I	MLP (2 114 params)	Breast Cancer Wisconsin (569)	Accuracy > 95%, F1 > 0.94
II	LeNet (61 706 params)	MNIST (70 000)	Accuracy = 98.41%
III	Seq2Seq GRU	EN→ES (≈6 000)	BLEU ≈ 1.0, exact ≈ 100%

Leçons apprises

1. **Adéquation architecture–données** : le CNN excelle sur les images grâce à l'invariance par translation et au partage de poids ; le MLP est efficace sur des features tabulaires indépendantes ; les architectures récurrentes capturent les dépendances séquentielles. Le choix de l'architecture doit être guidé par la **structure des données**.
2. **Importance du prétraitement** : la normalisation Z-score (MLP), les transformations image (CNN), et la tokenisation/padding (Seq2Seq) sont des étapes critiques qui conditionnent directement la convergence des modèles.
3. **Régularisation** : le Dropout (MLP), le gradient clipping (RNN) et la stratégie de checkpoint (MLP) sont essentiels pour stabiliser l'entraînement et améliorer la généralisation.
4. **PyTorch** : le framework offre une abstraction optimale pour implémenter rapidement des architectures variées via `nn.Module`, le GPU computing (`.to(device)`), et `torch.save/load`.
5. **Validation des implémentations** : la comparaison systématique entre implémentations manuelles et couches PyTorch natives (corrélation croisée, pooling) garantit la compréhension profonde des opérations.

Perspectives

Les pistes d'amélioration naturelles sont :

- **Mécanisme d'attention** (Bahdanau, 2014) pour lever le goulot d'étranglement du Seq2Seq
- **Architecture Transformer** (Vaswani *et al.*, 2017) basée entièrement sur l'attention et les connexions résiduelles
- **Transfer Learning** (fine-tuning de modèles pré-entraînés) pour améliorer les performances sur de petits jeux de données
- **Régularisation avancée** : Batch Normalization, Layer Normalization, Label Smoothing
- Sur le MLP Wine Quality, l'utilisation de méthodes ensemblistes (Random Forest, Gradient Boosting) servirait de base de comparaison

Projet réalisé dans le cadre du module Deep Learning.

EMSI Casablanca — Filière Informatique 4-IAD — Année universitaire 2025–2026